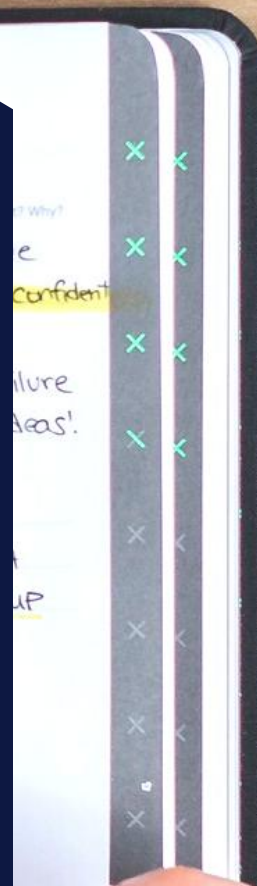




Práctica 6

Patrones de Diseño de
APIs

Elaborado por:
Jordan Murillo



Informe Individual — Práctica 06: Evaluación de Atributos de Calidad (ISO/IEC 25010) mediante Testing

Estudiante	Jordan Murillo
Carrera	Maestría de Software
Asignatura	Patrones de Diseño de APIs
N.º de práctica	06
Título	Evaluación de Atributos de Calidad (ISO/IEC 25010) mediante Testing
Fecha	25 de septiembre de 2026
Repositorio	Enlace

1. Objetivo

Evaluar dos atributos de calidad de ISO/IEC 25010 sobre la API REST de empleados (Express + TypeScript + MongoDB + Zod):

- **Modularidad y Capacidad de prueba:** probar el controlador aislado con mocks de Jest, sin mongoose.
- **Comportamiento temporal y Capacidad:** medir con Artillery el rechazo perimetral de Zod y la degradación del p99 bajo saturación.

2. Arquitectura implementada

Componente	Responsabilidad
<code>employee.controller.ts</code>	Controlador factory que recibe un <code>EmployeeRepository</code> inyectado.
<code>EmployeeRepository</code>	Interfaz del acceso a datos (<code>findAll</code> , <code>findById</code> , <code>create</code> , <code>update</code> , <code>delete</code>).
<code>MongooseEmployeeRepository</code>	Implementación real contra MongoDB (solo en producción).
<code>container.ts</code>	Contenedor de dependencias que inyecta el repositorio real.
<code>employee.controller.spec.ts</code>	Pruebas unitarias con el repositorio simulado con <code>jest.fn()</code> .
<code>stress-test.yml</code>	Escenario de Artillery: 20 s a 5 VUs/s y 30 s con rampa de 15 a 50 VUs/s; SLA p99 < 200 ms y <code>maxErrorRate</code> < 1 %.

Tabla 1. Componentes de la solución.

Cada VU envía 8 peticiones: 4 válidas ($201 + 3 \times 200$) y 4 corruptas (400). La prueba de estrés se ejecutó en dos entornos, cambiando solo `MONGO_URI`: **A** — MongoDB local y **B** — MongoDB Atlas.

3. Resultados obtenidos

3.1 Análisis de aislamiento (Jest)

`npm test:unit` terminó en verde: **10/10 pruebas en 0.225 s**, sin Docker ni MongoDB. Se cubren los casos exitosos (200, 201 con cabecera Location, 204), los 404 de negocio y la propagación de errores del repositorio al `errorMiddleware`.

Pregunta de control: ¿Por qué la Inversión de Dependencias permite probar el controlador sin Docker ni MongoDB?

Porque el controlador depende de la interfaz `EmployeeRepository` y no de su implementación. En la prueba, esa interfaz se cumple con funciones `jest.fn()`, y el controlador no distingue el mock de la clase real. Todo se ejecuta en memoria, sin conexiones de red.

3.2 Rendimiento colectivo (Artillery)

Métrica (consola)	A — Mongo local	B — MongoDB Atlas
VUs completados / fallidos	1075 / 0	1075 / 0
<code>http.codes.201 / 200 / 400</code>	1075 / 3225 / 4300	1075 / 3225 / 4300
<code>p50 / p95 / p99</code>	1 / 5 / 7.9 ms	76 / 3905.8 / 5826.9 ms
Mediana 4xx	1 ms	1 ms
Veredicto del ensure	PASA	FALLA

Tabla 2. Métricas de consola por entorno.

201 vs 400: los 400 ($4300 = 1075 \times 4$) coinciden exactamente con el tráfico corrupto inyectado. No hubo fugas a 2xx, rechazos de payloads válidos ni errores 5xx. Esto evidencia **Corrección funcional** del perímetro.

p99: en el Entorno A se mantiene en 7.9 ms, sin crecimiento exponencial. Si se forzara más la carga, los cuellos de botella síncronos del Event Loop serían:

- el logging de morgan;
- `JSON.parse` / `JSON.stringify`;
- `safeParse` de Zod;
- las consultas sin `.lean()`.

En el Entorno B el p99 sube a 5826.9 ms (FALLA). La causa es la latencia de red hacia Atlas, que produce encolamiento: la mediana es de 76 ms, pero el p99 es 76 veces mayor, y la sesión media sube de 7 s a 13.3 s. La mediana de los 4xx (1 ms) confirma que Zod rechaza el tráfico antes de tocar la base de datos.

4. Evidencia de funcionamiento

```
jmurillov1@JOPC:~/master/module-4/practica-06/backend$ pnpm test:unit
> backend@1.0.0 test:unit /home/jmurillov1/master/module-4/practica-06/backend
> NODE_OPTIONS=--experimental-vm-modules jest

(node:193874) ExperimentalWarning: VM Modules is an experimental feature and might change at any time
(Use `node --trace-warnings ...` to show where the warning was created)
PASS src/controllers/employee.controller.spec.ts
  Unit Test: EmployeeController (Mantenibilidad & Testabilidad)
    ✓ Debería retornar un estado 200 y la lista de empleados de la abstracción (2 ms)
    ✓ Debería retornar 201 y la cabecera Location al crear un empleado
    ✓ Debería propagar un AppError 404 cuando el empleado no existe (sin tocar la BD real) (1 ms)
  Casos adicionales: happy path
    ✓ getById: debería retornar 200 con el empleado cuando sí existe
    ✓ update: debería retornar 200 con el empleado actualizado
    ✓ remove: debería retornar 204 sin cuerpo cuando el empleado se elimina (1 ms)
  Casos adicionales: errores esperados
    ✓ update: debería propagar un AppError 404 si el empleado no existe (1 ms)
    ✓ remove: debería propagar un AppError 404 si el empleado no existe
    ✓ list: debería propagar el error del repositorio sin atraparlo (lo maneja el errorMiddleware) (7 ms)
    ✓ create: debería propagar el error del repositorio sin atraparlo (lo maneja el errorMiddleware) (1 ms)

Test Suites: 1 passed, 1 total
Tests: 10 passed, 10 total
Snapshots: 0 total
Time: 0.225 s, estimated 1 s
Ran all test suites.
jmurillov1@JOPC:~/master/module-4/practica-06/backend$ menos
```

Figura 1. pnpm test:unit: 10/10 pruebas en verde.

```
jmurillov1@JOPC:~/master/module-4/practica-06/backend$ pnpm stress:report
http.response_time.4xx:
  min: ..... 0
  max: ..... 38
  mean: ..... 0.9
  median: ..... 1
  p95: ..... 2
  p99: ..... 5
http.responses: ..... 8600
vusers.completed: ..... 1075
vusers.created: ..... 1075
vusers.created_by_name.Inundación transaccional perimetral (Payloads Mixtos ... 1075
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 7010.1
  max: ..... 7061.7
  mean: ..... 7019.9
  median: ..... 6976.1
  p95: ..... 6976.1
  p99: ..... 7117
Log file: artillery-report.json

Checks:
ok: maxErrorRate < 1
ok: http.response_time.p99 < 200
Veredicto: PASA
Reporte guardado en: /home/jmurillov1/master/module-4/practica-06/docs/historial/2026-09-26T03-23-04-304Z
Índice del historial: /home/jmurillov1/master/module-4/practica-06/docs/historial/index.html
jmurillov1@JOPC:~/master/module-4/practica-06/backend$ pnpm stress:report
```

Figura 2. Artillery — Entorno A (Mongo local). Veredicto: PASA.

```

-----
Summary report @ 22:30:40(-0500)
-----

http.codes.200: ..... 3225
http.codes.201: ..... 1875
http.codes.400: ..... 4300
http.downloaded_bytes: ..... 85808400
http.request_rate: ..... 170/sec
http.requests: ..... 8600
http.response_time:
  min: ..... 0
  max: ..... 7855
  mean: ..... 883.7
  median: ..... 76
  p95: ..... 3905.8
  p99: ..... 5826.9
http.response_time.2xx:
  min: ..... 72
  max: ..... 7855
  mean: ..... 1599.3
  median: ..... 1864.4
  p95: ..... 5168
  p99: ..... 5944.6
http.response_time.4xx:
  min: ..... 0
  max: ..... 287
  mean: ..... 8.2
  median: ..... 1
  p95: ..... 54.1
  p99: ..... 165.7
http.responses: ..... 8600
vusers.completed: ..... 1875
vusers.created: ..... 1875
vusers.created_by_name.Inundación transaccional perimetral (Payloads Mixtos ... 1875
vusers.failed: ..... 0
vusers.session_length:
  min: ..... 7316.3
  max: ..... 28304.5
  mean: ..... 13253.2
  median: ..... 14048.5
  p95: ..... 18588.1
  p99: ..... 19737.6
Log file: artillery-report.json

Checks:
ok: maxErrorRate < 1
fail: http.response_time.p99 < 200
Veredicto: FALLA (p99 (5826.9ms) > umbral del ensure (200ms))
Reporte guardado en: /home/jmurillov1/master/module-4/practica-06/docs/historial/2026-09-26T03-30-40-191Z
Índice del historial: /home/jmurillov1/master/module-4/practica-06/docs/historial/index.html
jmurillov1@J0PC:~/master/module-4/practica-06/backend$

```

Figura 3. Artillery — Entorno B (MongoDB Atlas). Veredicto: FALLA.

5. Conclusiones

- **La Inversión de Dependencias convierte la capacidad de prueba en una propiedad de la arquitectura, no del entorno.** El controlador depende de la interfaz `EmployeeRepository` y no de `Mongoose`, así que se validó con 10 pruebas en 0.225 s sin Docker ni MongoDB: casos exitosos, 404 de negocio y propagación de errores de infraestructura. Esto confirma la **Modularidad** y la **Capacidad de prueba** de ISO/IEC 25010.
- **La validación perimetral con Zod es exacta y barata.** En ambos entornos los rechazos 400 coinciden al 100 % con el tráfico corrupto inyectado (4300/4300), sin fugas a 2xx ni errores 5xx. La mediana de los 4xx es de 1 ms porque esas peticiones nunca llegan a la base de datos. Esto evidencia **Corrección funcional** y **Tolerancia a fallos** del perímetro.
- **El Comportamiento temporal depende de la infraestructura tanto como del código.** El mismo backend pasa de un p99 de 7.9 ms con Mongo local (PASA) a 5826.9 ms con Atlas (FALLA). La causa es la latencia de red y el encolamiento de conexiones, no la CPU del Event Loop: la degradación es no lineal, con un p99 76 veces mayor que la mediana.
- **El SLA solo puede afirmarse midiendo en un entorno equivalente al de producción.** Optimizar el Event Loop con `.lean()`, un logging más liviano y payloads más pequeños mejora el caso local. Contra una base remota, en cambio, hay que actuar sobre la **Capacidad**: desplegar backend y clúster en la misma región, dimensionar `maxPoolSize` y paginar el listado.

6. Referencias

- Repositorio del proyecto en GitHub: [Enlace](#).
- Prieto, P. (2026). *Guía de práctica: Evaluación de atributos de calidad (ISO/IEC 25010) mediante testing*. Universidad Politécnica Salesiana.
- ISO. (2023). *ISO/IEC 25010:2023 — Product quality model*.
- Jest. Mock Functions. <https://jestjs.io/docs/mock-functions>.
- Artillery. Documentation. <https://www.artillery.io/docs>.